

Videoflix: Schritte vom Localhost zum Hosting auf Render

Dieses Dokument beschreibt die konkreten Schritte, die unternommen wurden, um das „Videoflix“-Backend von einer lokalen Entwicklungsumgebung (localhost) auf Render zu hosten, einschließlich der Verbindung von Diensten und dem Hochladen von Videos.

1. Ausgangssituation (Localhost)

- **Backend-Setup:**
 - Django mit Django REST Framework (DRF) für die API.
 - PostgreSQL als lokale Datenbank für Benutzer- und Videodaten.
 - Redis für Caching ([django-redis](#)) und Task Queue ([django-rq](#)).
 - Gunicorn als Webserver.
 - Python-Version: 3.13.4.
- **Frontend:** Angular, gehostet auf [all-inkl.com](#) unter [anja-gollner.com](#).
- **Ziel:** Backend auf Render hosten mit PostgreSQL und Redis (Valkey), Videos über die Django-Admin-Oberfläche hochladen.

2. Schritte zum Hosting auf Render

2.1. Erstellung des Web Services

- **Aktion:**
 - Web Service [videoflix-backend-3zyr](#) im Render-Dashboard unter „New“ > „Web Service“ erstellt.
 - GitHub-Repository mit Django-Projekt verknüpft.
 - Startbefehl: [gunicorn videoflix.wsgi:application](#).
 - Port: 10000 (automatisch erkannt).
 - Umgebungsvariablen:

```
SECRET_KEY=dein-geheimer-schlüssel
DJANGO_ALLOWED_HOSTS=videoflix-backend-
3zyr.onrender.com,localhost,127.0.0.1,anja-gollner.com,www.anja-
gollner.com
CORS_ALLOWED_ORIGINS=https://anja-gollner.com
```

- [requirements.txt](#) enthält:

```
django==5.1.7
djangorestframework==3.15.2
django-rq==2.10.2
django-redis==5.4.0
dj-database-url==2.2.0
```

```
psycpg2-binary==2.9.9
unicorn==22.0.0
```

2.2. PostgreSQL-Datenbank einrichten

- **Problem:** Lokale PostgreSQL-Datenbank war nicht mit Render verbunden, was zu Fehlern wie `relation "auth_user" does not exist` führte.
- **Aktionen:**
 1. Im Render-Dashboard unter „New“ > „PostgreSQL“ eine Datenbank erstellt (Name: `videoflix-db`, Region: z. B. Frankfurt, Plan: Free).
 2. **Internal Database URL** kopiert (z. B. `postgres://user:password@host:port/dbname`).
 3. Umgebungsvariable im Web Service hinzugefügt:

```
DATABASE_URL=postgres://user:password@host:port/dbname
```

4. `settings.py` angepasst:

```
import dj_database_url
DATABASES = {
    'default': dj_database_url.config(
        default=os.getenv('DATABASE_URL'),
        conn_max_age=600
    )
}
```

5. Migrationen ausgeführt:

```
render run python manage.py migrate
```

- Erstellt Tabellen wie `auth_user` für Benutzer und `video_app_video` für Videos.

6. Superuser erstellt:

```
render run python manage.py createsuperuser
```

- Benutzername: `admin`, E-Mail: `admin@videoflix.com`, Passwort: `securepassword123`.

2.3. Redis (Valkey) einrichten

- **Problem:** Redis-Verbindung fehlgeschlagen (`Connection refused` auf `localhost:6379`), da keine lokale Redis-Instanz existierte.
- **Aktionen:**

1. Im Render-Dashboard unter „New“ > „Key Value“ eine Valkey-Instanz erstellt (Name: `videoflix-redis`, Region: z. B. Frankfurt, Plan: Free, Maxmemory Policy: `noeviction`).
2. **Internal URL** kopiert (z. B. `rediss://default:password@redis-123456.render.com:6379`).
3. Umgebungsvariable im Web Service hinzugefügt:

```
REDIS_URL=rediss://default:password@redis-123456.render.com:6379
```

4. `settings.py` angepasst:

```
REDIS_URL = os.getenv('REDIS_URL', 'redis://localhost:6379/0')
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": REDIS_URL,
        "OPTIONS": {"CLIENT_CLASS":
"django_redis.client.DefaultClient"},
        "KEY_PREFIX": "videoflix"
    }
}
RQ_QUEUES = {
    'default': {
        'URL': REDIS_URL,
        'DEFAULT_TIMEOUT': 60,
    }
}
```

5. Web Service redeployed:

```
render deploy
```

2.4. Medienspeicher für Videos und Thumbnails

- **Problem:** Videos und Thumbnails mussten gespeichert werden.
- **Aktionen:**
 1. Render-Disk erstellt:
 - Name: `media`, Mount Path: `/opt/render/project/src/media`, Größe: 10 GB.
 2. `settings.py` angepasst:

```
MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

3. `Haupt-urls.py` angepasst:

```
from django.conf import settings
from django.conf.urls.static import static
urlpatterns = [
    # ... andere URLs
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

2.5. Videos hochladen

- **Ziel:** Videos wie bei Netflix über die Admin-Oberfläche hinzufügen.
- **Aktionen:**
 1. `video_app/admin.py` konfiguriert:

```
from django.contrib import admin
from .models import Video
@admin.register(Video)
class VideoAdmin(admin.ModelAdmin):
    list_display = ('title', 'description', 'video_file',
        'category', 'created_at')
    list_filter = ('category', 'created_at')
    search_fields = ('title', 'description', 'category')
    ordering = ('-created_at',)
```

2. In der Admin-Oberfläche (<https://videoflix-backend-3zyr.onrender.com/admin/>) angemeldet mit Superuser (`admin / securepassword123`).
3. Videos hochgeladen unter „Video App“ > „Videos“ > „Add Video“:
 - Felder: `title`, `description`, `video_file` (MP4), `thumbnail` (optional), `category` (z. B. „action“).
 - Dateien in `media/videos/` und `media/thumbnails/` gespeichert.

2.6. Fehlerbehebung

- **CORS-Fehler:**
 - Problem: Frontend-Anfragen von `anja-gollner.com` wurden blockiert.
 - Lösung: `CORS_ALLOWED_ORIGINS` in `settings.py` gesetzt:

```
CORS_ALLOWED_ORIGINS = os.getenv('CORS_ALLOWED_ORIGINS',
    'https://anja-gollner.com').split(',')
```

- **DisallowedHost-Fehler:**
 - Problem: `videoflix-backend-3zyr.onrender.com` nicht in `ALLOWED_HOSTS`.
 - Lösung: `ALLOWED_HOSTS` in `settings.py` aktualisiert.
- **Datenbankfehler** (`relation "auth_user" does not exist`):
 - Lösung: Migrationen ausgeführt (`python manage.py migrate`).
- **Redis-Fehler** (`Connection refused`):
 - Lösung: Key-Value-Instanz erstellt und `REDIS_URL` verwendet.

- **400/500-Fehler bei `/api/registration/`:**
 - Lösung: Serializer geprüft, Migrationen ausgeführt, `DEBUG = True` für detaillierte Fehlermeldungen.

3. Ergebnis

- Das Backend wurde erfolgreich von localhost auf Render migriert.
- PostgreSQL und Valkey (Redis) wurden verbunden.
- Videos wurden über die Admin-Oberfläche hochgeladen und in der Render-Disk gespeichert.
- Die API (`/api/registration/`, `/api/videos/`) funktioniert für das Frontend auf `anja-gollner.com`.